

SRI SAIRAM COLLEGE OF ENGINEERING

BENGALURU

PROJECT REPORT

ON

AgriSense AI

AI-Powered Crop Production Prediction and Crop/Weed Detection System

Submitted by

Aishwarya Vantagodi

Department of Computer Science and Engineering

Sri Sairam College of Engineering, Bengaluru

Project / internship association: upskill Campus, The IoT Academy in association with UniConverge Technologies Pvt. Ltd. (UCT)

Academic Year: 2026

ABSTRACT

AgriSense AI is a web-based smart agriculture application designed to bring crop production prediction and image-based crop/weed detection together in one platform. The system provides a dashboard through which a user can enter agricultural information to obtain a crop production estimate or upload a field image for object detection.

The production-prediction workflow uses historical agricultural data and a trained regression model to estimate production for selected inputs. The image-analysis workflow uses a trained YOLO object-detection model to identify crop and weed objects in uploaded images and display detection results. A Python Flask application connects the user interface with these model workflows. The dashboard includes Overview, Production Prediction, Weed Detection, History, Analytics and About sections. SQLite is used for local record storage where configured.

This report describes the problem, objectives, requirements, system architecture, methodology, implementation, interface, testing approach, results, limitations and future enhancements. Actual model accuracy and performance metrics are not claimed where verified evaluation evidence has not been provided; such values should be added after formal testing.

ACKNOWLEDGEMENT

I express my sincere gratitude to Sri Sairam College of Engineering, upskill Campus, The IoT Academy and UniConverge Technologies Pvt. Ltd. (UCT) for the learning opportunity and support associated with this project. I also thank the faculty members, mentors and peers who provided guidance and encouragement. Their support helped me develop practical skills in Python, machine learning, computer vision and web application development.

TABLE OF CONTENTS

1. Introduction
2. Problem Definition and Objectives
3. Existing System and Proposed System
4. Requirement Analysis
5. System Architecture and Design
6. Methodology and Implementation
7. Dashboard and Module Description
8. Testing and Results
9. Limitations and Future Enhancements
10. Conclusion

11. References

Appendix A – Details to Complete

1. INTRODUCTION

1.1 Background

Agriculture depends on informed decisions about crop selection, production planning and field management. Crop production estimates can help users understand historical trends, while image-based weed identification can support field inspection. Machine learning and computer vision provide techniques for processing agricultural data and images.

AgriSense AI applies these technologies through a single web interface. The project combines a historicaldata-based crop production estimation workflow with a YOLO-based image detection workflow.

1.2 Project Overview

AgriSense AI is implemented as a Flask web application. Its main dashboard provides navigation to production prediction, weed detection, history and analytics. The application accepts two broad forms of input: structured agricultural values for production prediction and image files for crop/weed detection.

1.3 Scope

- Provide a web interface for crop production estimation using the configured historical dataset and trained regression model.
- Allow a user to upload a field image and run YOLO object detection.
- Display predicted values and image detection results through the dashboard.
- Provide summary statistics and visual analytics based on records available to the application.
- Maintain saved history using SQLite where the feature is enabled.

2. PROBLEM DEFINITION AND OBJECTIVES

2.1 Problem Statement

Crop production estimation and field-image weed identification are separate agricultural tasks. Users may need different tools to review historical production data and inspect field images. Manual weed identification can also require time and repeated visual inspection. The project addresses this gap by combining both workflows in a single application with a common dashboard.

2.2 Objectives

- Develop an integrated web application for crop production prediction and crop/weed image detection.
- Prepare and process historical agricultural data for model-based production estimation.
- Integrate a trained YOLO model for image-based object detection.
- Connect the models to a Flask backend and user-friendly frontend.
- Display prediction outputs, detection results and available activity summaries.
- Support review of saved records through History and Analytics pages.

2.3 Expected Outcomes

The expected outcome is a working prototype that demonstrates the integration of a regression-based production prediction workflow and a YOLO-based image detection workflow in a single web application. The project is intended as a decision-support prototype and does not replace agronomists or field-level validation.

3. EXISTING SYSTEM AND PROPOSED SYSTEM

3.1 Existing Approaches

Production estimates may be generated by analyzing historical agricultural records, while crop and weed objects may be identified using image-processing or computer-vision systems. When these functions are delivered separately, users may need to work across different tools and interfaces.

3.2 Proposed System

The proposed AgriSense AI system combines both functions in one Flask application. The production module accepts configured agricultural inputs and uses a trained regression model. The weed-detection module accepts an image and uses YOLO weights to generate object detections. The dashboard provides a common navigation and analytics layer.

3.3 Advantages

- Two agriculture-related AI workflows are accessible through one interface.
- Image detections can be presented with class labels and confidence values.
- The dashboard summarizes available activity through cards and charts.
- The application can be extended with additional datasets, models and agricultural services.

4. REQUIREMENT ANALYSIS

4.1 Hardware Requirements

Component	Requirement / note
Computer	Laptop or desktop capable of running Python and a web browser
Memory	Sufficient RAM for the Flask application and model inference; exact minimum depends on model size
Storage	Space for datasets, model weights, application files and uploaded images
Internet	Required for accessing a hosted deployment; local development can run without internet after dependencies are installed

4.2 Software Requirements

Software / technology	Purpose

Python	Main programming language
Flask	Backend web framework and routing
scikit-learn	Regression model training / inference, as configured
YOLO / Ultralytics	Object detection for uploaded field images
HTML, CSS, JavaScript	Frontend and dashboard
SQLite	Local database for records, where enabled
Visual Studio Code	Development environment
Web browser	Accessing the local or deployed application

4.3 Functional Requirements

- The application shall display the AgriSense AI overview dashboard.
- The user shall be able to access the Production Prediction page.
- The user shall be able to submit valid prediction inputs and view the generated estimate.
- The user shall be able to upload a supported image for weed/crop detection.
- The application shall display the detection output when model inference succeeds.
- The application shall provide History and Analytics views for available stored records.

4.4 Non-Functional Requirements

- Usability: navigation and forms should be understandable.
- Reliability: missing files and invalid inputs should be handled with clear feedback.
- Performance: model inference and page responses should be measured under representative conditions.
- Maintainability: frontend, backend, model and data files should be organized clearly.
- Security: uploads should be validated and sensitive configuration should not be hard-coded.

5. SYSTEM ARCHITECTURE AND DESIGN

5.1 High-Level Architecture

Layer	Production prediction path	Image detection path
Input	Crop/year or configured agricultural features	Uploaded field image
Preparation	Validate fields and prepare model features	Validate file and prepare image
Model	Trained regression model	Trained YOLO object detector
Backend	Flask route calls model and handles response	Flask route calls detector and handles response
Output	Estimated production value	Detected classes, confidence scores and annotated image
Storage	SQLite history, if enabled	SQLite detection history, if enabled

5.2 System Workflow

1. User opens the AgriSense AI dashboard.

2. User chooses Production Prediction or Weed Detection.
3. For production prediction, the application validates the submitted values and prepares the model input.
4. For image detection, the application validates the uploaded file and passes the image to the YOLO model.
5. The selected model generates an estimate or object detections.
6. The backend returns the output to the browser and stores records where database persistence is enabled. **5.3**

Data Flow

dashboard/history.

5.4 Database Design

SQLite is identified as the local database technology in the project. The application dashboard includes History and Analytics features, and the project code context describes local record storage. The exact table names, columns and relationships should be documented from the final database schema before submission; they are not assumed in this report.

6. METHODOLOGY AND IMPLEMENTATION

6.1 Development Environment

The application is developed using Python in Visual Studio Code. Flask provides the web server and route handling. The frontend is built with HTML, CSS and JavaScript. Machine-learning and computer-vision components are loaded by the backend for prediction and detection.

6.2 Crop Production Prediction Module

The production module is based on historical agricultural data. The general workflow is to load the dataset, inspect and clean relevant fields, prepare the input features expected by the selected regression algorithm, train and evaluate the model, save the trained model artifact, and load it within the Flask prediction route. When a user submits values through the web form, the backend prepares the same feature structure and returns the model estimate.

The exact dataset fields, selected final regression algorithm, training split and evaluation scores should be inserted from the final training script and recorded experiment. This report does not invent these details.

6.3 Weed Detection Module

The image-analysis module uses a YOLO object detector. The model is trained or fine-tuned using labeled agricultural images with the configured crop and weed classes. The trained weights are loaded by the application. During inference, the user uploads an image, the backend passes it to the detector, and the model returns object classes, confidence scores and bounding boxes. The application can display the annotated image and save the detection record where configured.

6.4 Flask Web Integration

Flask connects the browser interface with the model workflows. It serves dashboard pages, accepts form submissions and image uploads, invokes the relevant model code, and renders the output. The model files and dataset paths must match the application configuration for the service to run correctly.

6.5 Main Project Components

Component / file area	Responsibility
app.py	Flask application, routes, input handling and model integration
Model artifacts	Saved regression model and YOLO weights used for inference
templates/	HTML pages rendered by Flask
static/	CSS, JavaScript and frontend assets
Dataset / training scripts	Data preparation and model training resources
SQLite database	Local application records, where configured

7. DASHBOARD AND MODULE DESCRIPTION

7.1 Overview Dashboard

The Overview page is the main landing screen. The supplied dashboard screenshot shows the AgriSense AI

Explore Production Forecast and Analyze Field Image.

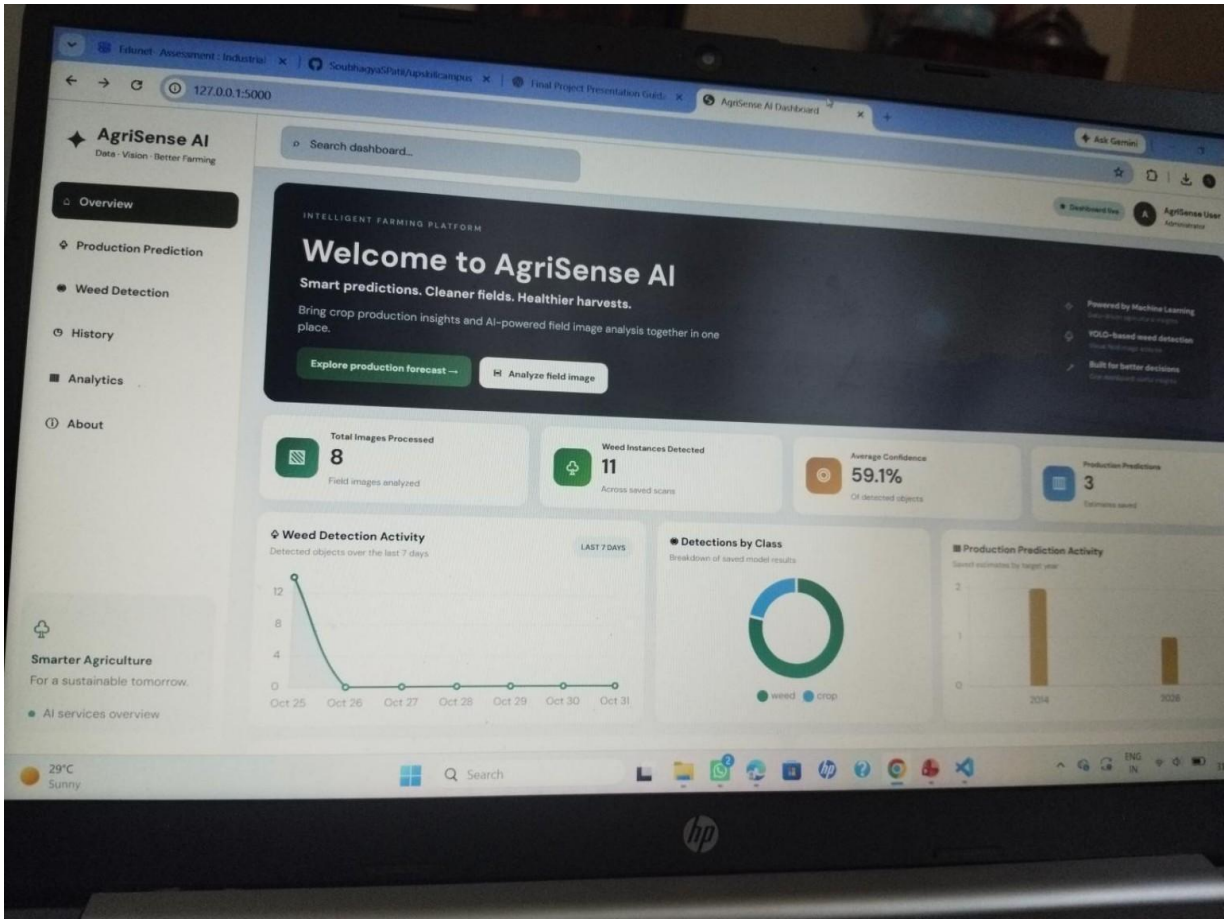


Figure 1: AgriSense AI overview dashboard captured from the running local application.

The screenshot displays four summary cards: Total Images Processed (8), Weed Instances Detected (11), Average Confidence (59.1%), and Production Predictions (3). These are the values visible in the provided screenshot at the time it was captured; they may change as the application records new activity.

The dashboard also shows Weed Detection Activity, Detections by Class (weed and crop), and Production Prediction Activity. These charts provide visual summaries of available saved records.

7.2 Production Prediction

The Production Prediction page allows the user to enter the agricultural values required by the configured model. After submission, the Flask backend sends the prepared input to the regression model and displays the estimated production result.

7.3 Weed Detection

The Weed Detection page allows the user to upload a field image. The YOLO model processes the image and returns detected object classes, confidence scores and bounding boxes. The result is displayed for user review.

7.4 History, Analytics and About

History: provides access to previously saved prediction and detection records, depending on database configuration.

Analytics: visualizes available records and activity summaries.

About: provides information about the project and its technologies.

8. TESTING AND RESULTS

8.1 Testing Approach

Testing should cover page navigation, valid and invalid form submissions, supported and unsupported image uploads, model inference, database persistence and chart consistency. Model quality should be evaluated separately using a held-out test dataset.

8.2 Test Cases

ID	Test	Expected result	Status
T01	Open overview page	Dashboard loads successfully	To be verified
T02	Navigate to Production Prediction	Prediction form opens	To be verified
T03	Submit valid prediction input	Model returns an estimate	To be verified
T04	Submit invalid or incomplete input	Validation feedback is shown	To be verified
T05	Upload supported image	Image is accepted for analysis	To be verified
T06	Upload unsupported file	File is rejected safely	To be verified
T07	Run YOLO detection	Detection output is displayed	To be verified
T08	Open History and Analytics	Available saved records are shown	To be verified

8.3 Observed Dashboard Values

The supplied dashboard screenshot visibly reports 8 total images processed, 11 weed instances detected, 59.1% average confidence, and 3 production predictions. These are interface counters and should not be interpreted as model accuracy or as a formal performance evaluation.

8.4 Model Evaluation

No verified model evaluation report was supplied for this project report. Before final submission, add the actual regression metrics (for example, MAE, RMSE or R^2 where applicable) and object-detection metrics (such as precision, recall and mAP) calculated on documented test data. Do not report confidence values as accuracy.

8.5 Limitations

Prediction quality depends on dataset completeness, relevance and feature quality.

Weed detection depends on image resolution, lighting, camera angle, class balance and label quality.

The dashboard counters depend on the records stored by the application.

The prototype requires formal end-to-end and field testing before being used for operational agricultural decisions.

9. LIMITATIONS AND FUTURE ENHANCEMENTS

9.1 Future Enhancements

Expand the historical dataset with verified regional and seasonal agricultural records.

Compare multiple regression algorithms and tune model parameters using suitable validation.

Collect more diverse crop and weed images and evaluate the detector with precision, recall and mAP.

Add weather, soil and location-related features when reliable data sources are available.

Add real-time field monitoring and IoT sensor integration.

Improve multilingual support, mobile responsiveness and user guidance.

Strengthen deployment, authentication, upload validation and error handling.

10. CONCLUSION

AgriSense AI demonstrates the integration of crop production prediction and image-based crop/weed detection in one web application. The Flask dashboard provides a common interface for entering agricultural data, uploading field images, viewing outputs and reviewing available analytics. The project combines Python, machine learning, YOLO-based computer vision and frontend web technologies to build an agriculturefocused prototype.

The work provides a foundation for further improvement through larger datasets, systematic model evaluation, field validation, improved user experience and additional agricultural data sources. The system should be treated as a prototype and its predictions should be validated before being used for real-world decisions.

11. REFERENCES

[1] Python Software Foundation, Python Documentation: https://docs.python.org/3/

[2] Pallets Projects, Flask Documentation: https://flask.palletsprojects.com/

[3] scikit-learn Documentation: https://scikit-learn.org/stable/

[4] Ultralytics, YOLO Documentation: https://docs.ultralytics.com/

[5] SQLite Documentation: https://www.sqlite.org/docs.html

[6] Project dashboard screenshot and project details supplied by the student.

[7] Internship template supplied for upskill Campus / The IoT Academy / UCT.

APPENDIX A – DETAILS TO COMPLETE BEFORE SUBMISSION

The following information was not fully available in the supplied project material. Complete it using the final source code, dataset, training logs and college requirements.

Student USN / registration number, faculty guide and project mentor names.

Project start/end dates and exact internship duration, if required.

Actual dataset name, source, fields, number of records and preprocessing steps.

Exact regression algorithm and evaluation metrics from the final training run.

YOLO model version, class names, dataset split and evaluation metrics.

GitHub repository URL and final deployment URL.

Screenshots of the Production Prediction form/output and Weed Detection input/output.

Executed test outcomes and any measured response-time results.